

pipeline

Builder visuel de pipelines CI/CD — Auto-hébergé

React 19

TypeScript

Fastify

MongoDB

Docker

Le problème

YAML complexe

Des dizaines de lignes de config pour un simple pipeline de déploiement

Zéro retour visuel

Impossible de voir la structure du pipeline sans l'exécuter

Erreurs silencieuses

Fautes de syntaxe ou jobs mal ordonnés découverts à l'exécution

```
.github/workflows/deploy.yml
```

```
name: Deploy to Production
on:
```

```
  push:
```

```
    branches: [main]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
    steps:
```

```
      - uses:
```

```
        actions/checkout@v4
```

```
      - name:
```

```
        Install dependencies
```

```
        run:
```

```
        npm ci
```

```
      - name:
```

```
        Build
```

```
        run:
```

```
        npm run build
```

```
      - name:
```

```
        Deploy
```

```
        uses:
```

```
...
```

Objectif du projet

Concevoir des pipelines CI/CD
visuellement, sans écrire une seule ligne de YAML.



Éditeur no-code

Drag-and-drop de stages, jobs et steps avec visualisation temps réel du graphe de dépendances.



Export multi-plateforme

Génération YAML pour GitHub Actions, GitLab CI, Azure DevOps et plus.



Auto-hébergé

Déployable sur n'importe quel serveur via Docker.
Données maîtrisées, zéro vendor lock-in.

Opportunités

Un marché dominé par la configuration manuelle

● **GitHub Actions / GitLab CI / Jenkins**
Outils config-first, courbe d'apprentissage élevée

● **n8n / Pipedream**
Visual-first mais pour l'automatisation, pas CI/CD

● **Codefresh / Harness**
Visual CI/CD mais SaaS, coûteux, vendor lock-in

pipel8ne se positionne comme

**la première solution
open-source, auto-hébergée
et visuelle pour CI/CD**

- Équipes DevOps cherchant simplicité
- PME sans expertise YAML avancée
- Projets open-source self-hosted
- Alternatives aux SaaS coûteux

Fonctionnalités clés



Éditeur drag-and-drop

Construction visuelle du graphe de pipeline via React Flow. Connexion de nœuds par glisser-déposer.



Hiérarchie Stage/Job/Step

Structure en 3 niveaux respectant les conventions CI/CD modernes. Validation en temps réel.



Export YAML

Génération du fichier de configuration pour la plateforme cible. Multi-format et extensible.



Gestion des credentials

Stockage chiffré des secrets. Accès par projet avec contrôle d'accès par rôle.



Gestion de projets

Organisation multi-projets avec hiérarchie et permissions. Tableau de bord centralisé.



Auth sécurisée

JWT access + refresh tokens. Sessions persistantes et révocation à la demande.

Stack technique

React 19

Frontend

TypeScript

Langage

Vite

Build

Tailwind CSS

Styles

React Flow

Éditeur graphique

Node.js

Runtime

Fastify

Framework HTTP

MongoDB

Base de données

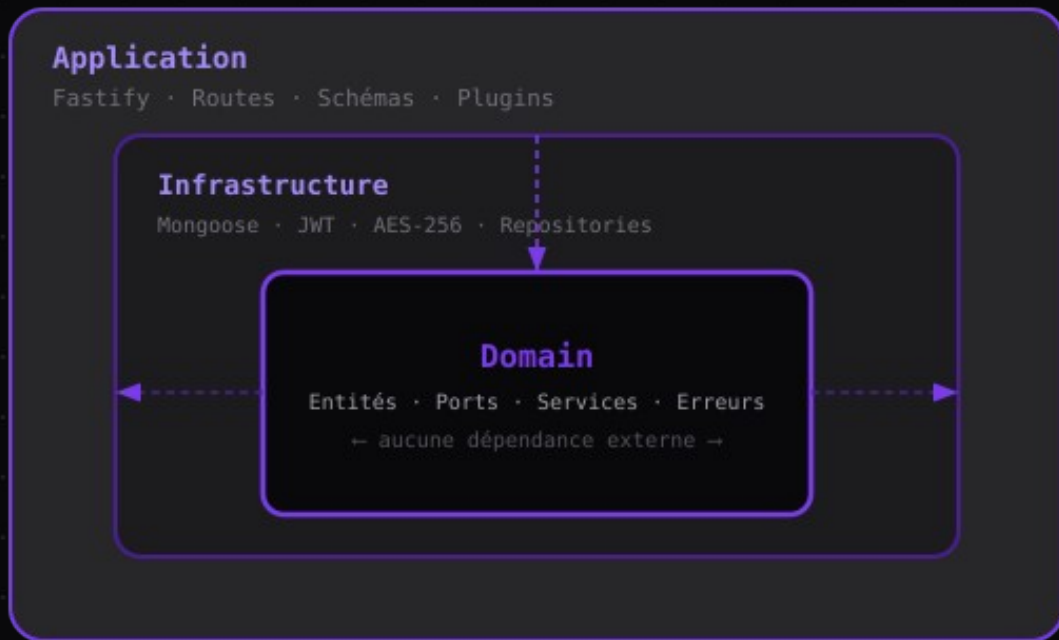
Docker

Conteneurisation

JWT + AES-256

Sécurité

Architecture hexagonale



Domain-first

Aucune dépendance externe dans le cœur métier. Testable en isolation.

Ports & Adapters

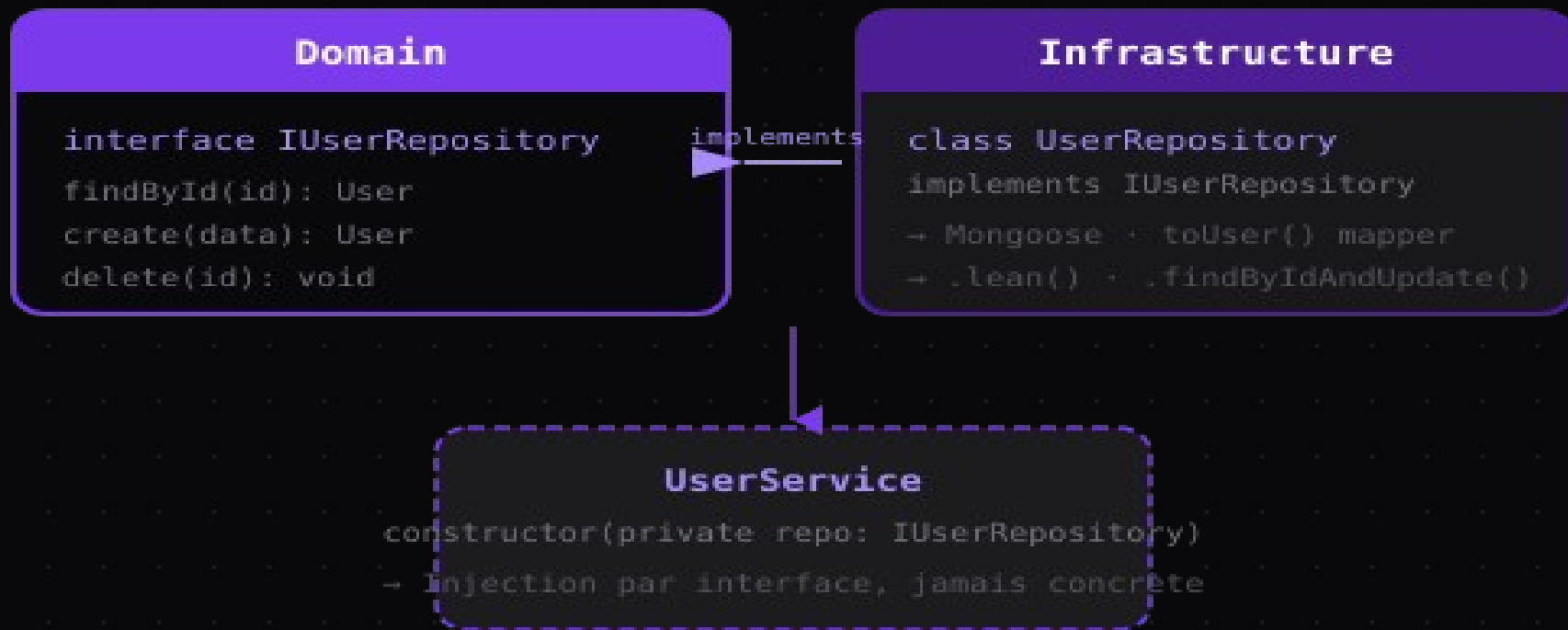
Interfaces (ports) définies dans le Domain, implémentées dans l'Infrastructure.

Flux unidirectionnel

Application → Domain ← Infrastructure.
Jamais dans le sens inverse.

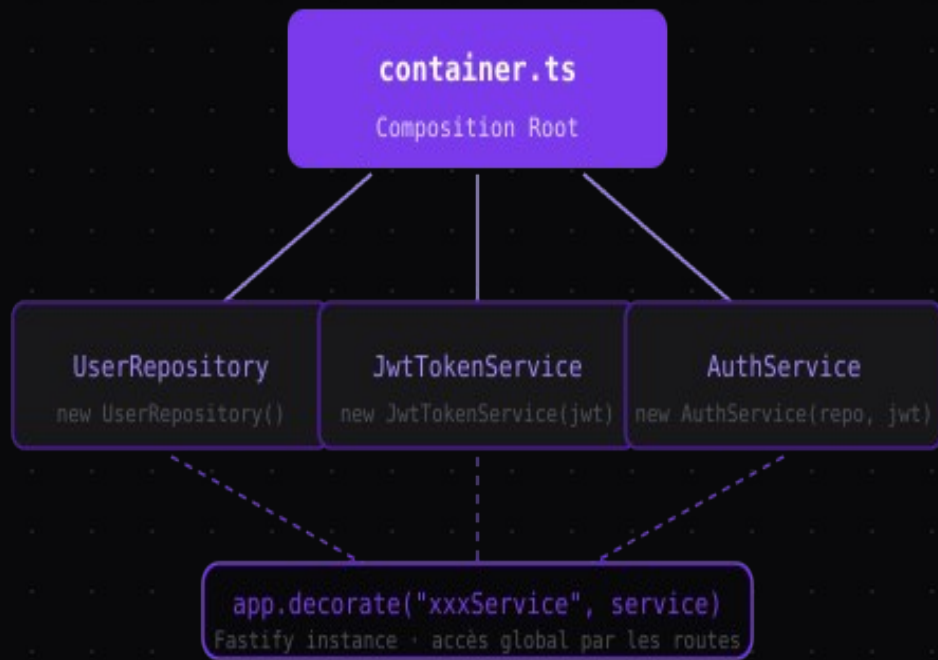
*Le Domain ne connaît pas l'Infrastructure.
L'Infrastructure ne connaît pas l'Application.*

Pattern Repository & Ports



Séparation totale entre l'interface (contrat) et l'implémentation concrète (Mongoose).

Injection de dépendances



Constructor injection

Chaque service déclare ses dépendances en paramètres de constructeur. Jamais d'instantiation dans les routes.

Composition Root

container.ts est le seul endroit où l'on instancie les repositories et services. Point d'entrée unique pour la DI.

Fastify decorators

Les services sont exposés via `app.decorate()` et disponibles dans tous les handlers comme `app.userService`.

Patterns frontend



Context API

`AuthContext`
`ThemeContext`

Context API

Etat global (auth, thème) sans prop drilling.
AuthContext: user courant, token, login/logout.



Custom Hook

`usePipeline`
Abstraction logique

Custom Hook

`usePipeline` encapsule la logique de l'éditeur:
nœuds, edges et callbacks React Flow.



API Layer

`Api/client.ts`
Couche HTTP centralisée

API Layer

`Api/client.ts` centralise les appels HTTP.
Séparation composants UI / transport réseau.



ProtectedRoute

HOC `guard`
Redirect si non-auth

Sécurité & Credentials

JWT

Access + Refresh
tokens

AES-256

Chiffrement
GCM des secrets

Argon2

Hachage des
mots de passe

OAuth2

Code dans
le Domain

Gestion des tokens JWT

- Access token court (15 min) — transmis en Authorization header
- Refresh token long (7j) — stocké en cookie HttpOnly
- Rotation automatique à chaque renouvellement
- Révocation possible côté serveur (blacklist)

Chiffrement des credentials

- SecretsService utilise AES-256-GCM (authenticated encryption)
- IV unique par chiffrement — résistance aux attaques replay
- Clé dérivée de APP_SECRET via l'environnement
- Déchiffrement uniquement au moment de l'utilisation

Roadmap



Prochaine priorité · Intégrations GitHub/GitLab/Azure DevOps · Sync directe de dépôt · Triggers webhook · Nouveaux types de steps